



VINUNIVERSITY

Spring 2025

COMP1020: Object-Oriented Programming & Data Structures

Lecture 2: Variables and Operators in Java

Expressions, Variables, Operators, Assignments and Statements.

Hieu Pham, College of Engineering & Computer Science, VinUniversity

February 20th, 2025

15-minute quiz

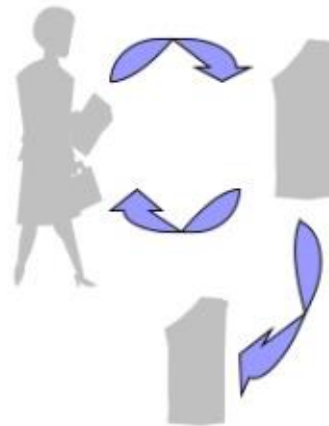
QUIZ 1 ACCESS CODE

20FEB25

LECTURE 1: RECAP

- Definition of POP and OOP, understand why and where you should use OOP instead of POP.
- Start thinking of your programming in terms of “objects” with classes, instead of a traditional “top down” programming approach.

Procedural vs Object-Oriented



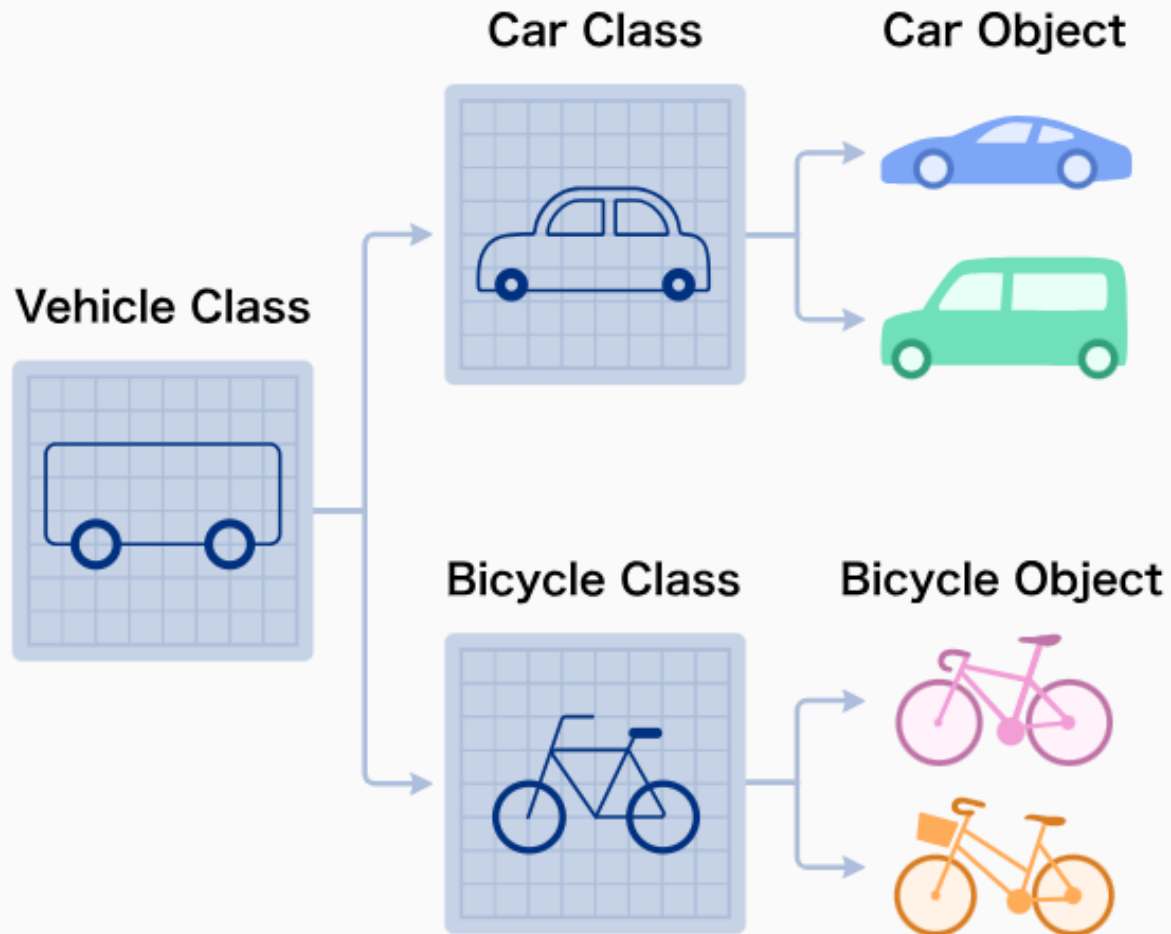
Withdraw, deposit, transfer



Customer, money, account

LECTURE 1: RECAP

Key principles and advantages behind OOP methodology.



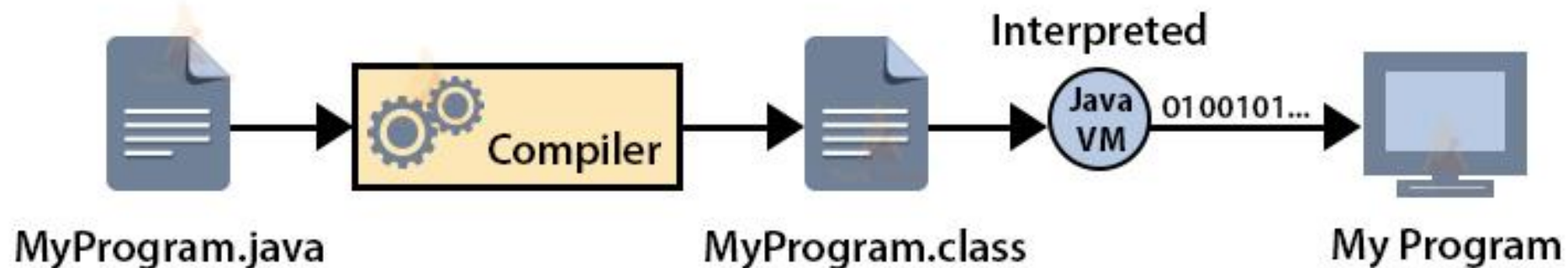
Example:

- **Encapsulation:** Each class contains attributes and methods specific to it.
- **Abstraction:** The Vehicle Class is a generalized concept that defines common attributes and behaviors of all vehicles.
- **Inheritance:** Car and Bicycle inherit from the Vehicle Class.
- **Polymorphism:** The `move()` and `stop()` methods apply to both Car and Bicycle.

How Java Virtual Machine works?

Java compiler (**javac**) compiles the java source code into the bytecode. **Java Virtual Machine** (JVM) then executes this bytecode which is executable on many operating systems.

Working of Java Virtual Machine



[[source](#)]

LECTURE 2: PLAN FOR TODAY

Learning outcomes

Upon the completion of this lecture, students will be able to:

- understand the **expressions, variables, operators, assignments, and statements** in Java.
- understand both **implicit and explicit castings** in Java. Here, casting refers to the process of converting one data type into another.

JAVA PROGRAM

- A Java program is a sequence of Java instructions (codes) that are executed in a certain order.
- All instructions must be located **inside a Java class** that may contain both **variables** (data) and **methods** (a set of operations on data).

```
public class MyClass{  
  
}
```

File name:

MyClass.java

Java extension

same name

PACKAGE AND HOW TO CREATE A PACKAGE IN JAVA?

What is a Package in Java?

- Package in Java is a mechanism to group related classes, sub-packages, and interfaces together.
- It helps in organizing code, avoiding class name conflicts and controlling access levels (public/private/protected/default).

Types of Java Packages

- Built-in packages: Predefined packages in Java (e.g., `java.util`, `java.io`, `java.lang`).
- User-defined package (create your own package's).

CREATING JAVA CLASS INSIDE A PACKAGE

To make a class as part of a package, you **have to select a name for your package and then include the package statement as the first statement in the source file.**

Include your
package here

```
// Name of package to be created  
package com.mycompany.mypackage;
```

```
// Class in which the above created package belongs to  
public class MyClass {  
    public void displayMessage() {  
        System.out.println("Hello from MyClass!");  
    }  
}
```



CREATING JAVA CLASS INSIDE A PACKAGE

To make a class as part of a package, you have to select a name for your package and then include the package statement as the first statement in the source file.

Define your
class with a class
name

```
// Name of package to be created  
package com.mycompany.mypackage;  
  
// Class in which the above created package belongs to  
public class MyClass {  
    public void displayMessage() {  
        System.out.println("Hello from MyClass!");  
    }  
}
```

CREATING JAVA CLASS INSIDE A PACKAGE

To make a class as part of a package, you have to select a name for your package and then include the package statement as the first statement in the source file.

Define your
class with a class
name

```
// Name of package to be created  
package com.mycompany.mypackage;  
  
// Class in which the above created package belongs to  
public class MyClass {  
    public void displayMessage() {  
        System.out.println("Hello from MyClass!");  
    }  
}
```

Your code

THE MAIN()

A Java program needs to start its execution from the **main** method of some class. The main method serves as the entry point for any standalone application.

Allows the method to be accessible from outside

Indicates that the method does not return any value.

Contains the supplied command-line arguments as an array of `String` objects.

```
public static void main(String[] args){  
  
}
```

Allows the method to be called without creating an instance of the class.

PUBLIC MODIFIER

- **public** is an access modifier, used to set **the access level for classes, attributes, methods, and constructors.**
- The class or method that is tagged as **public** can be used by **any other class** from any other package.
- Other access modifiers (will be covered later):
 -  **private**
 -  **default**
 -  **protected**

VOID IN JAVA

- When a method is declared with **void**, it performs its defined tasks but does not return any value back to the caller.

```
4 class Main {  
5     public static void main(String[] args) {  
6         System.out.println("COMP1020: OOP and Data  
           Structures");  
7     }  
8 }  
9 // Output: COMP1020: OOP and Data Structures
```

Use **void** when you need a method to perform actions like printing messages, modifying object states, or performing calculations without needing to return the result.

- If the method is **not void**, it can return any kind of data type.

```
static int myMethod(int x) {  
    return x + 10;  
}
```

JAVA NAMING CONVENTIONS

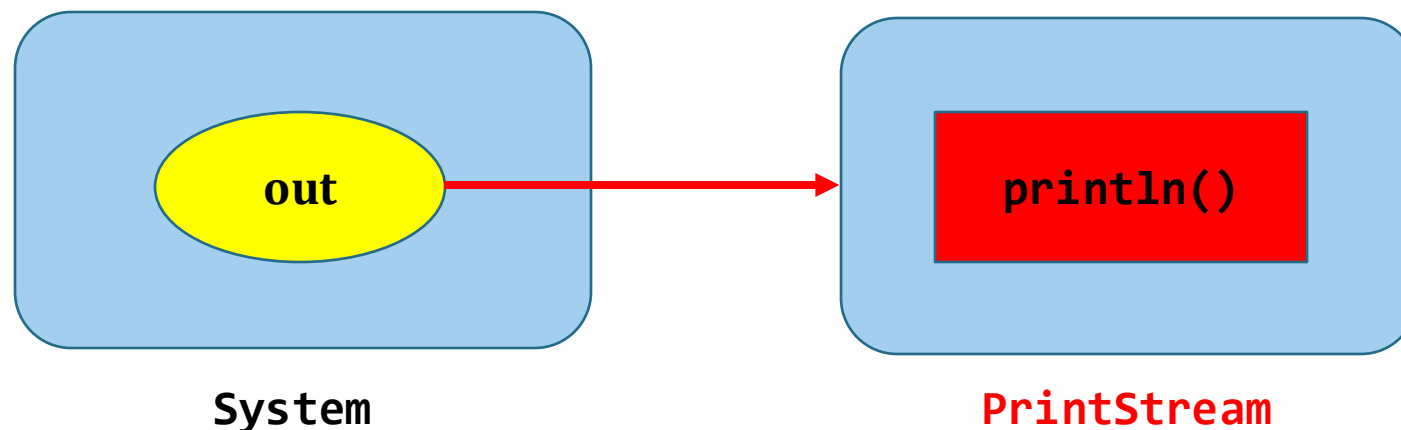
- Java variable names must start with **a letter**, the dollar sign (\$) or an underscore character (_).
- If the name you choose consists of only one word, spell that word in all lowercase letters.
- If the name is combined with two words, the second word will start with uppercase letter. For example: **countStudent()**, **printSalary()**, etc.

For more <https://docs.oracle.com/javase/tutorial/java/>

JAVA STANDARD OUTPUT: PRINT LINE

System.out.println

- The **System** is a class, **out** is the object of **PrintStream** class.
- The **println()** is the method of **PrintStream** class which displays the result and then moves the cursor to the next line.



[[source](#)]

JAVA STANDARD OUTPUT: PRINT LINE

Using `System.out.println` is straightforward. Just call it in your code with a value or an expression as a parameter, and it will print the result to the console.

```
public class Main {  
    public static void main(String[] args) {  
        int a = 5;  
        int b = 10;  
  
        System.out.println("Hello, world!");  
        System.out.println("The value of a is: " + a);  
        System.out.println("The sum of a and b is: " + (a + b));  
    }  
}
```

```
Hello, world!  
The value of a is: 5  
The sum of a and b is: 15
```



[Source](#)

CLASS PRACTICE

Printing Java Strings

What is the output of the following program?

```
1 // Online Java Compiler
2 // COMP1020 - Lecture 02
3 // How to use System.out.println()
4
5 class Main {
6     public static void main(String[] args) {
7         System.out.println(1 + 2 + "3" + 4 + 5);
8     }
9 }
```

CLASS PRACTICE

Printing Java Strings

What is the output of the following program?

```
1 // Online Java Compiler
2 // COMP1020 - Lecture 02
3 // How to use System.out.println()
4
5 class Main {
6     public static void main(String[] args) {
7         System.out.println(1 + 2 + "3" + 4 + 5);
8     }
9 }
```

Output: 3345

HELLO WORLD!

Writing your first program in Java.

```
1 package spring.OOP;  
2  
3 public class MyFirstJava {  
4  
5     public static void main(String[] args) {  
6         // TODO Auto-generated method stub  
7         System.out.println("Hello World!");  
8     }
```

Java
Block

COMPILE AND RUN

The screenshot shows the Eclipse IDE interface. On the left, the 'Package Explorer' displays a project named 'HelloWorld' with a source folder 'src' containing a package 'spring.OOP' and a class 'MyFirstJava'. A context menu is open over 'MyFirstJava', with the 'Run As' option selected and highlighted by a red circle. The 'Run As' submenu is also visible, showing 'Java Application' and 'Run Configurations...'. The main editor displays the following Java code:

```
1 package spring.OOP;  
2  
3 public class MyFirstJava {  
4  
5     public static void main(String[] args) {  
6         // TODO Auto-generated method stub  
7         System.out.println("Hello World!");  
8     }  
9  
10 }  
11
```

At the bottom, the 'Console' view shows the output of the program:

```
<terminated> MyFirstJava [Java Application] C:\Users\preskokseng\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_14.0.2.v20200815-0932\jre\bin\javaw.exe (Feb  
Hello World!
```

JAVA VARIABLE TYPES

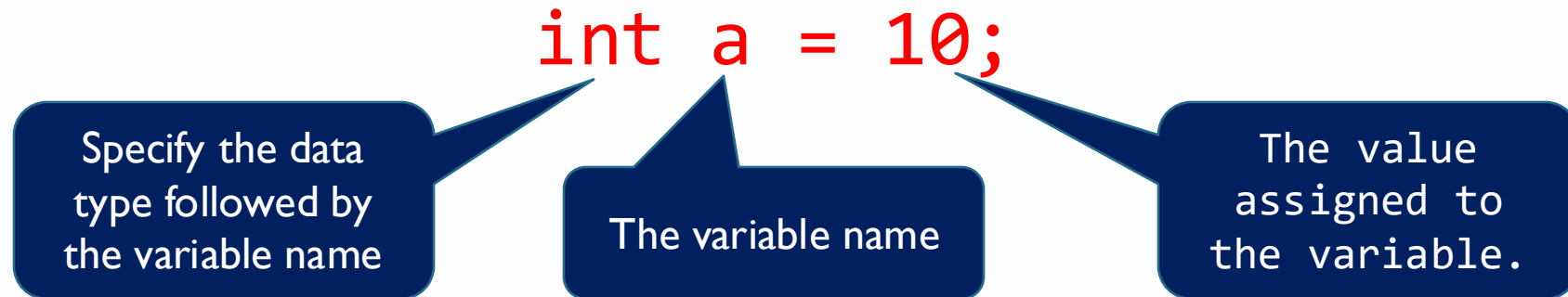
Variables are containers for storing data values. In Java, there are different types of variables:

- **Non-static fields (instance fields)** – belongs to an object.
- **Static fields (class fields)** – variable that belongs to a class.
- **Local variables** – variable declared inside a method.
- **Parameters** – variable that is passed to a method when the method is called/executed.

Note that all variables must first be declared before they can be used!

PRIMITIVE VS REFERENCE

- **Primitive Variable** is basic type of variables that store a single value. It is used to represent primitive values, e.g.,



PRIMITIVE VS REFERENCE

Reference Variable: Unlike primitive variables which hold values directly, reference variables are used to store **the memory address of objects** rather than the actual data.

Let create a class called
“Car” with its properties.

Then create a method to
display information about
the Car.

```
class Car {  
    // Instance variables (attributes of the Car)  
    String name;  
    String model;  
    int year;  
  
    // Constructor method to initialize the Car object  
    public Car(String name, String model, int year) {  
        this.name = name;  
        this.model = model;  
        this.year = year;  
    }  
  
    // Method to display information about the Car  
    public void displayInfo() {  
        System.out.println("Name: " + name);  
        System.out.println("Model: " + model);  
        System.out.println("Year: " + year);  
    }  
}
```

PRIMITIVE VS REFERENCE

Reference Variables: Unlike primitive variables which hold values directly, reference variables are used to store the memory address of objects rather than the actual data.

Now we will create an Object of the above class.

```
public class Main {  
    public static void main(String[] args) {  
        // Create a Car object using the Car class  
        Car myCar = new Car("Toyota", "Camry", 2022);  
  
        // Access and display information about the Car object  
        System.out.println("Car Information:");  
        myCar.displayInfo();  
    }  
}
```

myCar is a reference variable that refers to the object of **Car** class stored in the memory address.

PRIMITIVE DATA TYPES IN JAVA

Primitive Data Types: specifies the size and type of variable values.

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

JAVA FLOAT

- Used to save memory in large arrays of floating-point numbers.
- Default value (the initial value that the variable takes if it is declared as a class field but not explicitly initialized.): 0.0f

```
10.0  
10.59  
10.1  
5.812167E8  
3.4028235E22
```

```
public class MyFloatType {  
  
    public static void main(String[] args) {  
        float f1 = 10;  
        float f2 = 10.59f;  
        float f3 = 10.1f;  
        float f4 = 581216732.323433f;  
        float f5 = 3.40282346638528860e+22f;  
  
        System.out.println(f1);  
        System.out.println(f2);  
        System.out.println(f3);  
        System.out.println(f4);  
        System.out.println(f5);  
    }  
}
```

OBJECT TYPES IN JAVA

- **Object Data Types**: used to store objects and are referenced by object reference variables.
- Non-primitive data type.
- Created by the users in Java.

JAVA EXPRESSIONS

Java expressions consist of variables, operators, literals and method calls.

```
int age;  
age = 22;
```

An expression that
returns an int

```
int currentYear = 2021;  
int bornYear = currentYear - age;  
System.out.println(bornYear);
```

An expression

```
if (bornYear > 1990)  
    System.out.println("You are young!");
```

An expression

JAVA STATEMENTS

- A Java statement is a complete unit of execution.
- Java statements must end with a semicolon (;).

Semicolon is used to indicate the end of a command, statement, or declaration.

```
double price = 0.65 * 7;
```

Declaration
statement

This statement involves multiplying float 0.65 and integer 7, and then assigning the result to the variable price

JAVA COMMENTS

- **Single-line comment** starts with two forward slashes (`//`).

```
// TODO Auto-generated method stub
```

- **Mutli-line comment** starts with (`/*`) and ends with (`*/`).

```
/* to print the price  
double price = 0.65 * 7;  
System.out.println(price);  
*/
```

- **Javadoc** comments start with (`/**`) and ends with (`*/`).

```
/**  
 * @param firstName  
 * @param lastName  
 * @param hourlyRate  
 */
```

JAVA OPERATORS

- Data in Java is manipulated using **operators**.
- An operator takes one or more arguments and produces a new value:
 - addition and unary plus (+)
 - subtraction and unary minus (-)
 - multiplication (*)
 - division (/)
 - assignment (=)

```
int number1 = 15, number2 = 9;

//Addition Operation
int sum = number1 + number2;
System.out.println("Sum is: " + sum);

//Subtraction Operation
int dif = number1 - number2;
System.out.println("Difference is : " + dif);

//Multiplication Operation
int mul = number1 * number2;
System.out.println("Multiplied value is : " + mul);

//Division Operation
int div = number1 / number2;
System.out.println("Quotient is : " + div);

//Modulus Operation
int rem = number1 % number2;
System.out.println("Remainder is : " + rem);
```

```
Sum is: 24
Difference is : 6
Multiplied value is : 135
Quotient is : 1
Remainder is : 6
```

STRING CONCATENATION

In java, string concatenation forms a new string that is the combination of multiple strings.

```
String str1 = "Hello";  
String str2 = "World";  
String str3;  
  
str3 = str1.concat(str2);  
System.out.println(str3);  
System.out.println(str1 + str2);  
System.out.println(str1 += str2);
```

Outputs?

STRING CONCATENATION

In java, string concatenation forms a new string that is the combination of multiple strings.

```
String str1 = "Hello";  
String str2 = "World";  
String str3;
```

```
str3 = str1.concat(str2);  
System.out.println(str3);  
System.out.println(str1 + str2);  
System.out.println(str1 += str2);
```

Outputs:

HelloWorld
HelloWorld
HelloWorld

PRECEDENCE OF OPERATORS IN JAVA

In Java, operator precedence determines the order in which operators are evaluated in an expression.

Operators	Precedence
!, +, - (unary operators: ++a / a++)	Highest
*, /, % (multiplication, division, modulus)	2 nd
+, - (addition, subtraction)	3 rd
<, <=, >=, >	4 th
!= (not equal to)	5 th
&& (logical AND)	6 th
(logical OR)	7 th
= (assignment operator)	Lowest

OPERATORS PRECEDENCE EXAMPLE

```
double discountPercent = .2;  
double price = 100;  
price = price * 1 - discountPercent;  
System.out.println(price);
```

What is the output?

OPERATORS PRECEDENCE EXAMPLE

```
double discountPercent = .2;  
double price = 100;  
price = price * 1 - discountPercent;  
System.out.println(price);
```

What is the output?

Answer: 99.8

UNARY OPERATORS

In Java, unary operators are operators that operate **on only one operand**. They are used to perform simple operations such as **incrementing, decrementing, negating values, or checking boolean conditions**.

Operator	Name	Example Expression	Description
++	Postfix increment	<code>i++</code>	add 1 to i and return the old value
	Prefix increment	<code>++i</code>	add 1 to i and return the new value
--	Postfix decrement	<code>i--</code>	take 1 from i and return the old value
	Prefix decrement	<code>--i</code>	take 1 from i and return the new value

CLASS PRACTICE

Sample Code:

```
public static void main(String []args){  
    int x=5;  
    System.out.println(x++);  
    System.out.println(++x);  
    System.out.println(x++);  
    System.out.println(x);  
}
```

Output:



CLASS PRACTICE

Sample Code:

```
int a = 9;  
int b = 9;  
int c = ++a;  
int d = b++;  
System.out.println(a);  
System.out.println(b);  
System.out.println(c);  
System.out.println(d);
```

Output:



CLASS PRACTICE

Sample Code:

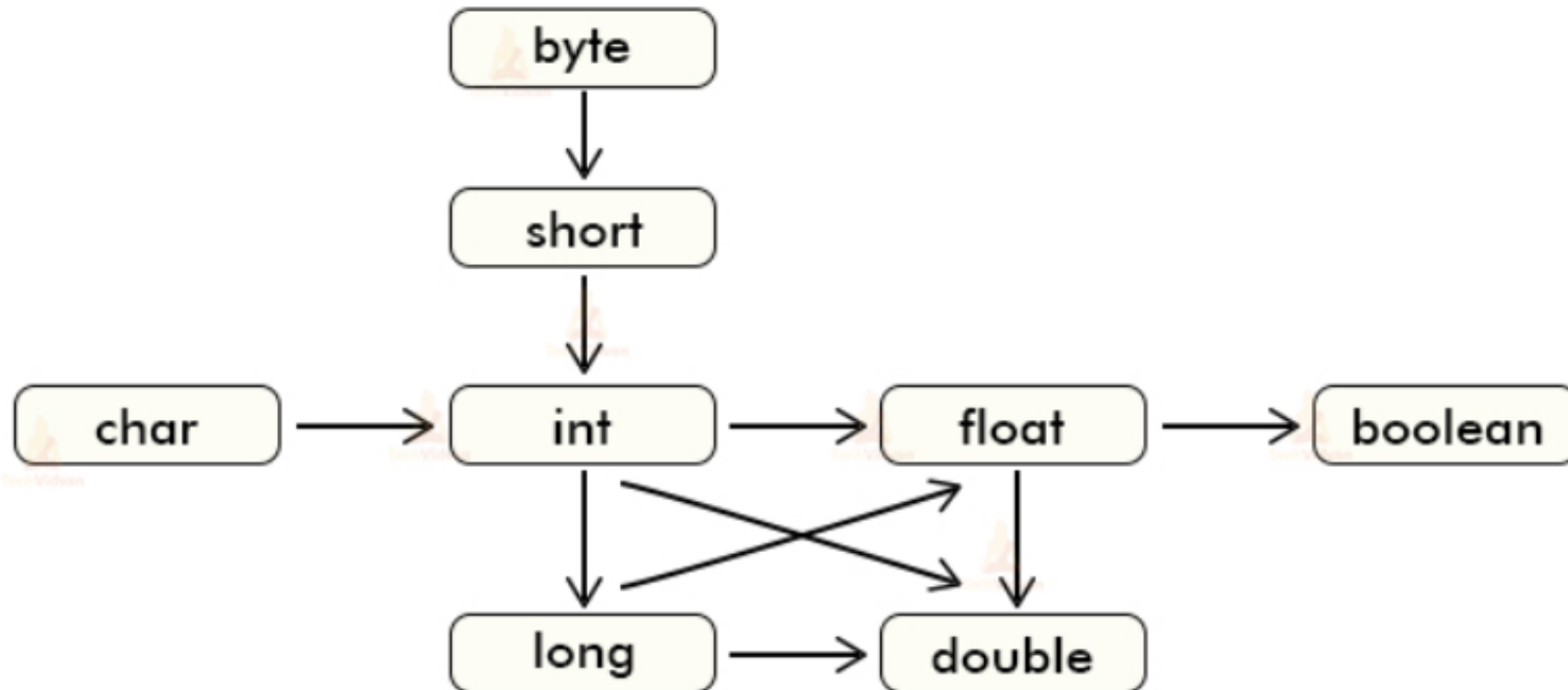
```
int p = 6;  
System.out.println("p=: " + p++);  
System.out.println("p=: " + p);  
  
int q = 6;  
System.out.println("q=: " + q--);  
System.out.println("q=: " + q);  
  
int r = 6;  
System.out.println("r=: " + ++r);  
  
int s = 6;  
System.out.println("s=: " + --s);
```

Output:



JAVA CASTING

Type Casting: the process of converting a variable of one predefined type into another.



[[source](#)]

IMPLICIT CASTING

Implicit Casting (also known as **Widening Casting**): two data types are automatically converted when

1. They are compatible (same data types).
2. We assign the value of a smaller data type to a larger data type.

byte → short → char → int → long → float → double

```
double doubleScore = 12.322;  
float floatScore = doubleScore;
```

A valid casting?

Exception in thread "main" java.lang.Error: Unresolved compilation problem:
Type mismatch: cannot convert from double to float

IMPLICIT CASTING EXAMPLE

convert int to
double

```
double finalExam = 93;  
double midtermExam = 91.0;  
int quiz = 74, project = 85;  
double average = (finalExam + midtermExam + quiz + project)/4;  
System.out.println("Your average score is " + average);
```

Convert quiz and
project to double

EXPLICIT CASTING

- **Explicit Casting** (also known as **Narrowing Casting**): a process of explicitly converting a type to a specific type manually.
- It involves converting a larger data type to a smaller data type.

double → float → long → int → char → short → byte

- Useful for incompatible data types, where automatic conversion cannot be performed.

EXPLICIT CASTING EXAMPLE

```
double doubleValue = 3.7142;
```

```
//all the digits after the decimal are lost
```

```
int intValue1 = (int)doubleValue;
```

Syntax: (typename) expression;

```
//built-in function to convert double to integer
```

```
int intValue2 = (int)Math.round(doubleValue);
```

```
System.out.println(intValue1);
```

```
System.out.println(intValue2);
```

accepts a double value
and converts it into
nearest long value

GET INPUT(S) FROM USER

- Java uses **System.out** (refer to the standard output device) and **System.in** (refer to the standard input device).
- Package: **java.util**
- Class: **Scanner**
- Common methods:



`next()`



`nextLine()`



`nextInt()`

```
import java.util.Scanner;
```

Import java.util package

```
public class MyPrintString {
```

How about next()?

```
public static void main(String[] args) {
```

```
Scanner input = new Scanner(System.in);
```

Creating a new scanner object

```
System.out.println("Enter your name: ");
```

```
String name = input.nextLine();
```

Calling scanner method nextLine()

```
System.out.print("How many thieves are watching now? ");
```

```
int numb = input.nextInt();
```

Calling scanner method nextInt()

```
System.out.println("The return of " + name + " and the " + numb + " thieves!");
```

```
// closing the scanner object
```

```
input.close();
```

```
}
```

Closing the scanner object

Enter your name:

Ali Baba

How many thieves are watching now? 40

The return of Ali Baba and the 40 thieves!

```
1 import java.util.Scanner; // Scanner is in the j
2
3 public class ComputeAreaWithConsoleInput {
4     public static void main(String[] args) {
5         // Create a Scanner object
6         java.util.Scanner input = new java.util.Scanner(System.in);
7
8         // Prompt the user to enter a radius
9         System.out.print("Enter a number for radius:
10        double radius = input.nextDouble();
11
12        // Compute area
13        double area = radius * radius * 3.14159;
14
15        // Display results
16        System.out.println("The area for the circle
17        radius + " is " + area);
18    }
19 }
```

The **import** statement can be omitted if you replace **Scanner** by **java.util.Scanner**

JAVA SCANNER CLASS METHODS

Method	Description
<code>int nextInt()</code>	It is used to scan the next token of the input as an integer.
<code>float nextFloat()</code>	It is used to scan the next token of the input as a float.
<code>double nextDouble()</code>	It is used to scan the next token of the input as a double.
<code>byte nextByte()</code>	It is used to scan the next token of the input as a byte.
<code>String nextLine()</code>	Advances this scanner past the current line.
<code>boolean nextBoolean()</code>	It is used to scan the next token of the input into a boolean value.
<code>long nextLong()</code>	It is used to scan the next token of the input as a long.
<code>short nextShort()</code>	It is used to scan the next token of the input as a Short.
<code>BigInteger nextBigInteger()</code>	It is used to scan the next token of the input as a BigInteger.
<code>BigDecimal nextBigDecimal()</code>	It is used to scan the next token of the input as a BigDecimal.

[[source](#)]



THANK YOU!

Lecture 3: Program structure, organization, and modular programming in Java.